

# Optimising Elevator Algorithms - Can we do better?

A First Principles Exploration

Lim Jia Wei

2026-01-04

## 1. The Vertical Commute

My campus dormitory, Saga College, is a wonderfully made 22-story building overlooking the UTown Green. While the architecture is impressive, the lifts have historically been a major point of contention within our community. Students living on the highest levels are often compensated with great views and cooler weather, but those perks are frequently overshadowed by the frustration of long waiting times. During peak periods, it is not uncommon to wait up to five minutes for a lift; a ridiculous delay when you are rushing for a class.

To combat these unequal wait times, the community developed an implicit social contract: the “non-consecutive floor” policy. The idea is simple: if you live on the 5th floor and the 4th or 6th floor has already been pressed, you take the initiative to walk one flight of stairs rather than forcing the lift to stop again. While this arrangement benefits the community’s overall efficiency, it remains a point of friction. Every year, seniors find themselves “lecturing” the incoming batch on lift etiquette, trying to bridge the gap between individual convenience and collective speed.

While this has been the center of many community debates - this problem (at least in my own view) isn’t just a social one, but it also invites a deeper analysis into the analysis of elevator algorithms in general. Other than social conditioning, *how might one efficiently program a lift to ensure all students are serviced fairly without the “Stop-and-Go” cycles that lead to these exceedingly long waits?*

This article is an exploration of lift algorithm design—from simple simulations to the current frontier of the field—to see if we can close the gap between academic theory and the daily reality of community living.

## 2. Lift Algorithms at its Core: How does a Lift “Think”?

The problem I have described is classified as a stochastic optimisation problem. It is “stochastic” because passenger arrivals are random and unpredictable, and it is an “optimisation” problem because we are interested in finding the best way to utilise a limited resource—in this case, our two lift cars—to service a 22-story building.

How might we go about comparing lift algorithms to one another to quantitatively prove that one algorithm is more efficient than another? We first need a way to measure the “cost” of a lift’s journey.

## 2.1 Modelling the Lift Cycle

In a simplified model, the efficiency of a lift is described by two factors: how far it travels and how many times it stops. We can model the total time of a single journey ( $T$ ) using a linear equation:

$$T = (t_f \cdot D) + (t_s \cdot S)$$

**Where:**

- $t_f$ : The constant time taken to pass a single floor without stopping.
- $D$ : The total distance (number of floors) the lift travels.
- $t_s$ : The Stop Penalty—the fixed time lost for doors opening, passenger transfer, and doors closing.
- $S$ : The number of stops made during the journey.

While  $D$ ,  $t_s$ , and  $t_f$  are physical constants,  $S$  is a discrete random variable dependent on passenger demand. Consequently, the total journey time  $T$  becomes a function of the expected number of stops  $E[S]$ .

$$E[T] = (t_f \cdot D) + (t_s \cdot E[S])$$

The assumption that  $t_s \gg t_f$  underscores the sensitivity of the model and implies  $E[T]$  is dominated by  $S$ . In this context, the “non-consecutive floor” policy is a community-driven attempt to keep  $E[S]$  as low as possible.

## 2.2 The SCAN Algorithm

The SCAN algorithm, often known simply as the “Elevator Algorithm,” is a classic piece of logic that dates back to the 1980s. It was designed for an era where computer memory was expensive, so the instructions had to be as simple as possible. (Wikipedia contributors 2024)

The defining characteristic of SCAN is its **naive travel direction**. We define the lift’s position at any time  $t$  as  $f(t) \in \{1, 2, \dots, 22\}$ . The direction of travel  $d(t)$  is a binary state that only switches when the lift hits the terminal floors:

$$d(t+1) = \begin{cases} 1(\text{Up}) & \text{if } f(t) = 1 \\ -1(\text{Down}) & \text{if } f(t) = 22 \end{cases}$$

Because the lift must complete the full  $(1 \rightarrow 22 \rightarrow 1)$  regardless of demand, this implies that  $D = 2 \cdot D_{max}$ . Consequently, the average waiting time,  $E[T_{scan}]$ , under the SCAN algorithm is dependent solely on  $E[S]$ :

$$E[T_{scan}] = (2 \cdot t_f \cdot D_{max}) + (t_s \cdot E[S])$$

### 2.3 The LOOK Algorithm: A Smarter Variant?

Most modern elevators utilise the LOOK algorithm, which improves upon SCAN by enabling the lift to “smartly” “look” ahead, and reverse direction as soon as no further requests exist in the current headed direction. For a set of active floor requests  $R$ , we define the upper and lower bounds of this set as:

$$r_{max} = \max(r \in R), \quad r_{min} = \min(r \in R)$$

The direction  $d(t)$  only reverses if the lift reaches these dynamic bounds:

$$d(t+1) = -d(t) \quad \text{if} \quad \nexists r \in R \text{ s.t.} \quad \begin{cases} r > f(t) & \text{if } d(t) = 1 \\ r < f(t) & \text{if } d(t) = -1 \end{cases}$$

We can express the expected average waiting time under the LOOK algorithm,  $E[T_{look}]$ , by the expected range of these requests:

$$E[T_{look}] = \underbrace{(2 \cdot t_f \cdot E[r_{max} - r_{min}])}_{\text{Expected Travel Time}} + \underbrace{(t_s \cdot E[S])}_{\text{Expected Stop Penalty Time}}$$

#### 2.3.1 The Limitation of “Smart” Logic

In the theoretical limit where passenger load  $P \rightarrow \infty$ , the LOOK algorithm’s “smartness” becomes obsolete as it converges perfectly to the SCAN baseline. In this saturated state, the probability of an early reversal vanishes, and the expected distance travelled by the elevator in half a cycle tends to the building’s maximum height:

$$\lim_{P \rightarrow \infty} Pr(r_{max} < N) = \lim_{P \rightarrow \infty} \left( \frac{N-1}{N} \right)^P = 0$$

$$\lim_{P \rightarrow \infty} E[r_{max} - r_{min}] = N - 1 = D_{max}$$

$$\therefore \lim_{P \rightarrow \infty} E[T_{look}] = (2 \cdot D_{max} \cdot t_f) + (t_s \cdot E[S]) = E[T_{scan}]$$

Real-world elevators, thankfully, have only finite capacity. My building’s lift happens to have a maximum load of  $P = 15$ . In this case, the probability that the lift is forced to the terminal floor is essentially a coin flip:

$$Pr(r_{max} = 22) = 1 - \left(\frac{21}{22}\right)^{15} \approx 0.504$$

Even at a more realistic “peak” load of say  $P = 7$ —I personally have yet to encounter a time when the lift was so full that I couldn’t even enter—there still remains a  $\approx 28\%$  chance that at least one passenger has requested the 22nd floor.

On the surface, it seems like the theoretical limit was too pessimistic. If the lift reverses early 72% of the time ( $P = 7$ ) at the busiest times of the day, then the LOOK algorithm should, in practice, significantly outperform SCAN by minimising  $E[T]$ .

While this appears to be a reasonable conclusion, this argument assumes that  $D$  is the dominant source of delay. However, our model underscores that the time cost of a journey is dependent on the time spent moving and the time spent stopping:

$$E[T] = \underbrace{(t_f \cdot E[D])}_{\text{Expected Travel Time}} + \underbrace{(t_s \cdot E[S])}_{\text{Expected Stop Penalty Time}}$$

### 2.3.2 The Distance vs Stops Trade-Off

The “trap” in modern elevator logic lies in our initial assumption that that  $t_s \gg t_f$ . We can define the Stop-to-Transit Ratio ( $\rho$ ):

$$\rho = \frac{t_s}{t_f}$$

By formalizing this ratio, we arrive at the fundamental governing principle of the system:

#### **Proposition 1 (The Stop-Equivalence Principle)**

In a system where  $\rho = t_s/t_f$ , the temporal cost of a single stop  $S$  is equivalent to the cost of traversing  $\rho$  floors of distance  $D$ .

$$\Delta T(S) \approx \Delta T(D_\rho)$$

Where  $\rho$  represents the “Effective Floor Distance” of the stop penalty.

To verify this proposition, we consider the conditions required for the LOOK algorithm to outperform a manual stop-reduction strategy. For the algorithm to “win,” the floors saved by reversing early ( $\Delta D$ ) must outweigh the penalty of the stops made:

$$\Delta D \cdot t_f > S \cdot t_s \quad \implies \quad \Delta D > S \cdot \rho$$

The dominance of the stop count is best illustrated by the partial derivatives of the  $T$ .

$$\frac{\partial T}{\partial S} = t_s, \quad \frac{\partial T}{\partial D} = t_f$$

$$\therefore \frac{\partial T}{\partial S} = \rho \cdot \frac{\partial T}{\partial D}$$

Because  $t_s$  accounts for the entire procedure of a lift stopping at a certain floor—including deceleration, door operation, and passenger transfer—it is typical for  $\rho \gg 1$  in most modern day elevators.

To put this into perspective, if  $\rho \approx 10$  (i.e., stopping takes 10s vs. passing a floor takes 1s), a single extra stop is equivalent to adding **10 floors of travel distance**.

### 2.3.3 The Asymmetry of Bounded Savings

We compare the maximum possible benefit of the algorithm ( $\Delta D_{max}$ ) against the cumulative cost of the stops ( $C_{stops}$ ).

1. The Hard Ceiling on Savings. The LOOK algorithm saves time by reversing at the highest requested floor ( $r_{max}$ ) instead of traveling all the way to the top floor ( $N$ ). The distance saved is the round-trip travel between those two points:

$$\Delta D = 2(N - r_{max})$$

Because the lift cannot travel higher than the roof ( $N$ ) or lower than the lobby (1), this value is physically constrained. Even in the theoretical “best case” scenario (where the algorithm somehow avoids going to the top entirely), the savings are strictly bounded by the total height of the building:

$$\Delta D < 2N$$

In a 22-story building, the algorithm has a hard limit of 44 floors. It physically cannot save more distance than exists in the shaft.

2. The High Ceiling on Penalties. Conversely, the “Effective Distance” added by stops scales linearly with the number of interruptions  $S$ , which is bounded by the lift capacity  $P = 15$ . Using our arbitrary equivalence ratio  $\rho \approx 10$ :

$$C_{stops} = S \cdot \rho = 10S$$

Unlike the distance savings, this penalty has no hard ceiling. As long as people keep entering the lift,  $S$  keeps increasing.

3. The Inevitable Intersection. We can calculate the Saturation Point ( $S_{crit}$ ) where the penalty from stops completely overwhelms the maximum possible theoretical savings of the algorithm:

$$\underbrace{S \cdot \rho}_{\text{Penalty}} > \underbrace{2N}_{\text{Max Possible Savings}}$$

$$S > \frac{2N}{\rho}$$

Substituting the values for Saga College ( $N = 22, \rho = 10$ ):

$$S > \frac{44}{10} = 4.4$$

This inequality provides a stark mathematical proof: At just 5 stops, the “virtual distance” added by the passengers exceeds the total physical height of the building. Even if the LOOK algorithm performs perfectly—saving the maximum possible distance every single time—its contribution is mathematically erased if the passengers make just 5 stops. The system is fighting a losing battle: the algorithm is optimized to save “inches” (distance) while the passengers are adding “miles” (stops).

As such, community efforts to avoid consecutive floor pressing is a high-impact intervention. While the algorithm is designed to optimise the path ( $D$ ), the residents are the only ones capable of optimising the variable that actually drives the delay: the number of stops ( $S$ ). By coordinating to avoid consecutive floors, students are solving the heavy part of the equation that the elevator’s software cannot.

### 3. Architectural Redesign

Talk about how two lifts solve the problem

### **3.1 Destination Dispatch**

Inefficiencies for super tall buildings and their concept of grouping individuals before hand

### **3.2 Grouping and Dynamic Zoning**

reserving dedicated lifts to serve certain floors only and how we can dynamically allocate those zones depending on varying frequency of demand

## **4. Algorithmic Optimization**

asdas

### **4.1 Branch and Price Frontier**

asdasd

### **4.2 Branch and Bound (Pruning)**

adasd

## **References**

Wikipedia contributors. 2024. “Elevator Algorithm — Wikipedia, the Free Encyclopedia.” [https://en.wikipedia.org/wiki/Elevator\\_algorithm](https://en.wikipedia.org/wiki/Elevator_algorithm).